protobuf

Protocol Buffers - Google's data interchange format

Thrift

PB vs. Thrift vs. Avro

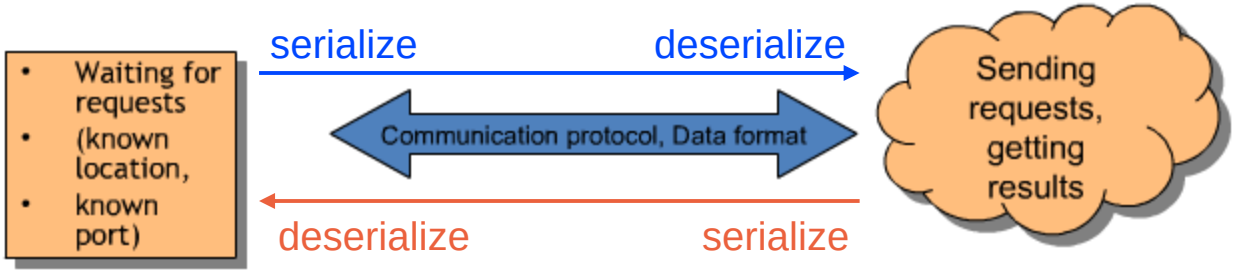
AVRO™

Author: Igor Anishchenko

Lohika - May, 2012

Problem Statement

Simple Distributed Architecture



- **Basic questions are:**
  - What kind of protocol to use, and what data to transmit?
  - Efficient mechanism for storing and exchanging data
  - What to do with requests on the server side?

…and you want to scale your servers...

- When you grow beyond a simple architecture, you want..
  - flexibility
  - ability to grow
  - latency
  - and of course - ***you want it to be simple***

How components talk

- Database protocols - fine.
- HTTP + maybe JSON/XML on the front - cool.

How components talk

- Database protocols - fine.
- HTTP + maybe JSON/XML on the front - cool.
- But most of the times you have internal APIs.

Hasn't this been done before? (yes)

- SOAP
- CORBA
- DCOM, COM+
- JSON, Plain Text, XML

Should we pick up one of those? (no)

- **SOAP**
  - XML, XML and more XML. Do we really need to parse so much XML?
- **CORBA**
  - Amazing idea, horrible execution
  - Overdesigned and heavyweight
- **DCOM, COM+**
  - Embraced mainly in windows client software
- **HTTP/JSON/XML/Whatever**
  - Okay, proven – hurray!
  - But lack protocol description.
  - You have to maintain both client and server code.
  - You still have to write your own wrapper to the protocol.
  - XML has high parsing overhead.
  - (relatively) expensive to process; large due to repeated tags

Decision Time?

As a developer - what are you looking for?

Be patient, I have something for you on the subsequent slides!!

High level goals!

- **Transparent interaction between multiple programming languages**
  - A language and platform neutral way of serializing structured data for use in communications protocols, data storage etc.

High level goals!

- Transparent interaction between multiple programming languages
  - A language and platform neutral way of serializing structured data for use in communications protocols, data storage etc.
- **Maintain Right balance between:**
  - Efficiency (how much time/space?)
  - Ease and speed of development
  - Availability of existing libraries and etc..

Consideration: Protocol Space

{"deposit_money": "12345678"}	
JSON	Binary
'0x6d', '0x6f', '0x6e', '0x65', '0x79', '0x31', '0x32', '0x33', '0x34', '0x35', '0x36', '0x37', '0x38'	'0x01', '0xBC614E'

Binary takes less space. No contest!

Consideration: Protocol Time

JSON	Binary
Push down automata (PDA) parser (LL(1), LR(1)) -- 1 character lookahead. Then, final translation from characters to native types (int, float, etc)	No parser needed. The binary representation **IS** [as close as to] the machine representation.

Binary is way faster. No contest

Consideration: Protocol Ease of Use

JSON	Binary
Brainless to learn Popular	Need to manually write code to define message packets (total pain and error prone!!!) **or** Use a code generator like Thrift (oh noes, I don't want to learn something new!)

Json is easier, binary is a pain.

Several smart people have attacked this problem over the years and as a result there several good open source alternatives to choose from

Here is where
Data Interchange Protocols
comes in play…

Serialization Frameworks

XML, JSON,

Protocol Buffers, BERT,

BSON, **Apache Thrift**, Message Pack,

Etch, Hessian, ICE, **Apache Avro**,

Custom Protocol...

SF have some properties in common

- Interface Description (IDL)
- Performance
- Versioning
- Binary Format

Protocol Buffer

- Designed ~2001 because everything else wasn't that good those days
- Production, proprietary in Google from 2001-2008, open-sourced since 2008
- Battle tested, very stable, well trusted
- Every time you hit a Google page, you're hitting several services and several PB code
- PB is the glue to all Google services
- Official support for four languages: C++, Java, Python, and JavaScript
- Does have a lot of third-party support for other languages (of highly variable quality)
- Current Version - protobuf-2.4.1
- BSD License

protobuf
Protocol Buffers - Google's data interchange format

Apache Thrift

- Designed by an X-Googler in 2007
- Developed internally at Facebook, used extensively there
- An open Apache project, hosted in Apache's Inkubator.
- Aims to be the next-generation PB (e.g. more comprehensive features, more languages)
- IDL syntax is slightly cleaner than PB. If you know one, then you know the other
- Supports: C++, Java, Python, PHP, Ruby, Erlang, Perl, Haskell, C#, Cocoa, JavaScript, Node.js, Smalltalk, OCaml and Delphi and other languages
- Offers a stack for RPC calls
- Current Version - thrift-0.8.0
- Apache License 2.0

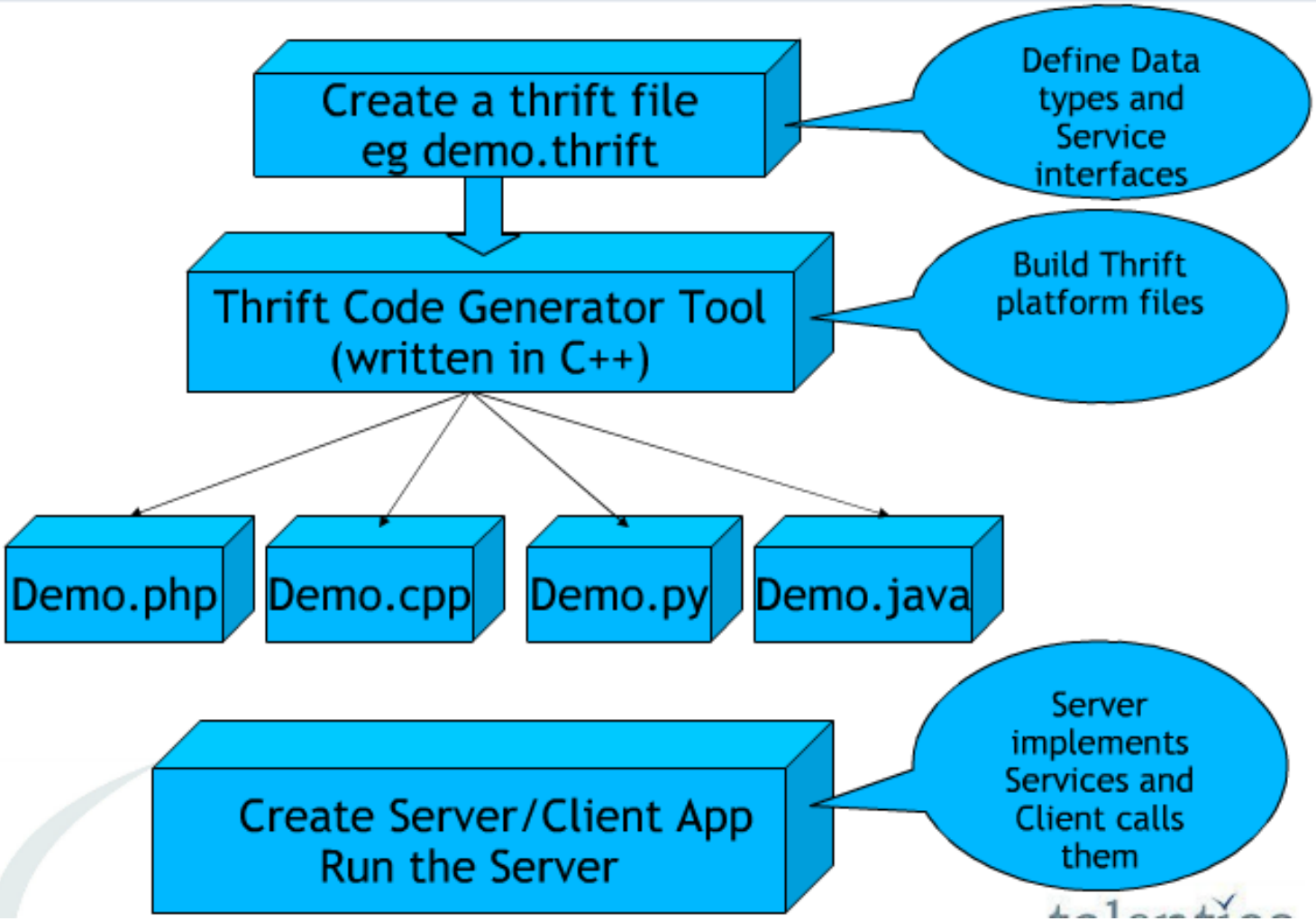
Avro

- I have a lot to say about **Avro** towards the end

Typical Operation Model

- The typical model of Thrift/Protobuf use is
  - Write down a bunch of struct-like message formats in an IDL-like language.
  - Run a tool to generate Java/C++/whatever boilerplate code.
    - Example: *thrift --gen java MyProject.thrift*
  - Outputs thousands of lines - but they remain fairly readable in most languages
  - Link against this boilerplate when you build your application.
  - DO NOT EDIT!

Thrift Principle of Operation



Interface Definition Language (IDL)

- Web services interfaces are described using the *Web Service Definition Language*. Like SOAP, WSDL is a XML-based language.
- The new frameworks use their own languages, that are not based on XML.
- These new languages are very similar to the *Interface Definition Language*, known from CORBA.

Thrift

```
namespace java serializers.thrift.media

typedef i32 int
typedef i64 long

enum Size {
  SMALL = 0,
  LARGE = 1,
}
enum Player {
  JAVA = 0,
  FLASH = 1,
}

struct Image {
  1: string uri, //url to the images
  2: optional string title,
  3: required int width,
  4: required int height,
  5: required Size size,
}

struct Media {
  1: string uri, //url to the thumbnail
  2: optional string title,
  3: required int width,
  4: required int height,
  5: required list<string> person,
  6: required Player player,
  7: optional string copyright,
}

struct MediaContent {
  1: required list<Image> image,
  2: required Media media,
}
```

Protobuf

```
package serializers.protobuf.media;

option java_package = "serializers.protobuf.media";
option java_outer_classname = "MediaContentHolder";
option optimize_for = SPEED;  affects the C++ and Java
code generators

message Image {
  required string uri = 1; //url to the thumbnail
  optional string title = 2; //used in the html
  required int32 width = 3; // of the image
  required int32 height = 4; // of the image
  enum Size {
    SMALL = 0;
    LARGE = 1;
  }
  required Size size = 5;
}

message Media {
  required string uri = 1;
  optional string title = 2;
  required int32 width = 3;
  required int32 height = 4;
  repeated string person = 5;
  enum Player {
    JAVA = 0;
    FLASH = 1;
  }
  required Player player = 6;
  optional string copyright = 7;
  }

message MediaContent {
  repeated Image image = 1;
  required Media media = 2;
}
```

Defining IDL Rules

- Every field **must** have a unique, positive integer identifier ("= 1", " = 2" or " 1:", " 2:")
- Fields may be marked as ’required’ or ’optional’
- structs/messages may contain other structs/messages
- You may specify an optional "default" value for a field
- Multiple structs/messages can be defined and referred to within the same .thrift/.proto file

Tagging

- The numbers are there for a reason!
- The "= 1", " = 2" or " 1:", " 2:" markers on each element identify the unique "tag" that field uses in the binary encoding.
- It is important that these tags do not change on either side
- Tags with values in the range 1 through 15 take one byte to encode
- Tags in the range 16 through 2047 take two bytes
- Reserve the tags 1 through 15 for very frequently occurring message elements

Java Example (Thrift example)

```
// this file is BankDeposit.thrift
struct BankDepositMsg {
    1: required i32 user_id;
    2: required double amount = 0.00;
    3: required i64 datestamp;}
```

```
...
import bank_example.BankDepositMsg;
...
BankDepositMsg my_transaction = new BankDepositMsg();
my_transaction.setUser_id(123);
my_transaction.setAmount(1000.00);
my_transaction.setDatestamp(new Timestamp(date.getTime()));
...
```

In Java (and other compiled languages) you have the getters and the setters, so that if the fields and types are erroneously changed the compiler will inform you of the mistake.

The Comparison…

	Thrift	Protocol Buffers
Composite Type	Struct {}	Message {}
Base Types	bool byte 16/32/64-bit integers double string	bool 32/64-bit integers float double string byte sequence
Containers	**list<t1>:** An ordered list of elements of type t1. May contain duplicates. **set<t1>:** An unordered set of unique elements of type t1. **map<t1,t2>:** A map of strictly unique keys of type t1 to values of type t2.	No
Enumerations	Yes	Yes
Constants	Yes *Example:* const i32 INT_CONST = 1234; const map<string,string> MAP_CONST = {"hello": "world", "goodnight": "moon"}	No
Exception Type/Handling	Yes (exception keyword instead of the struct keyword.)	No

The Comparison

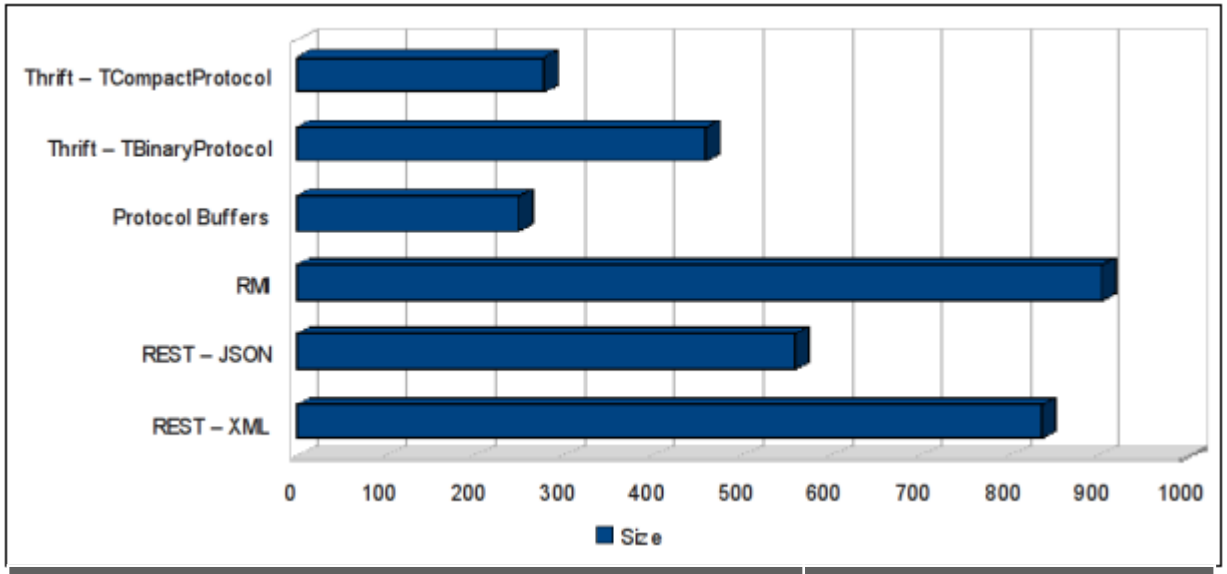
	Thrift	Protocol Buffers
License	Apache	BSD-style
Compiler	C++	C++
RPC Interfaces	Yes	Yes
RPC Implementation	Yes	No (they do have one internally)
Composite Type Extensions	No	Yes
Data Versioning	Yes	Yes

Performance

- To keep things simple a lot is missing in the new frameworks.
- For example the extensibility of XML or the splitting of metadata (header) and payload (body).
- Of course the performance depends on the used operating system, programming language and the network.
- Size Comparison
- Runtime Performance

Size Comparison

Each write includes one Course object with 5 Person objects, and one Phone object.



Method	Size (smaller is better)
Thrift — TCompactProtocol	278 (not bad)
Thrift — TBinaryProtocol	460
Protocol Buffers	250 (winner!)
RMI	905
REST — JSON	559
REST — XML	836

TBinaryProtocol – not optimized for space efficiency. Faster to process than the text protocol but more difficult to debug.

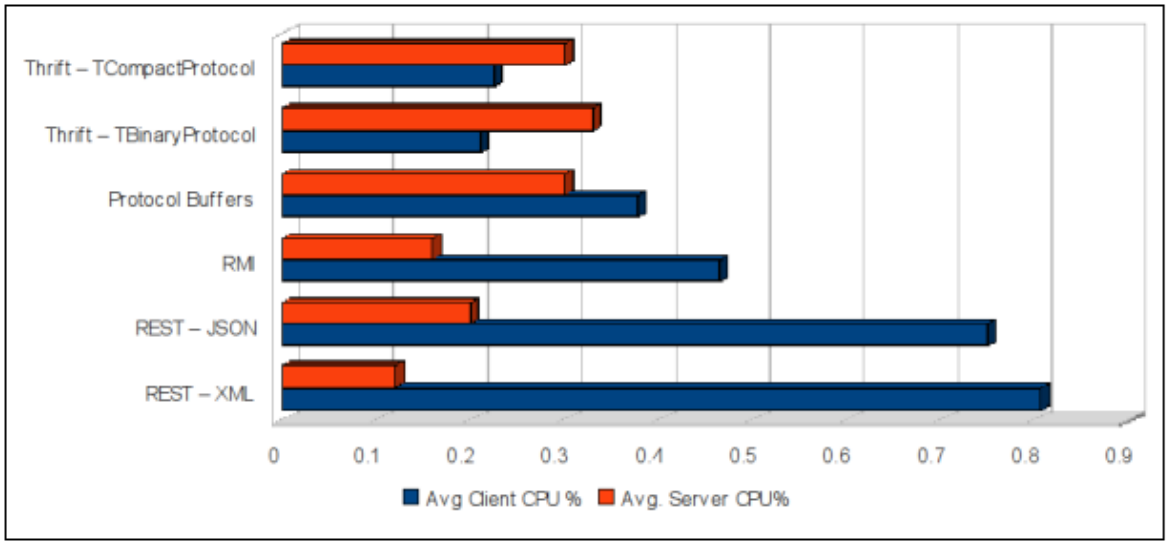
TCompactProtocol – More compact binary format; typically more efficient to process as well

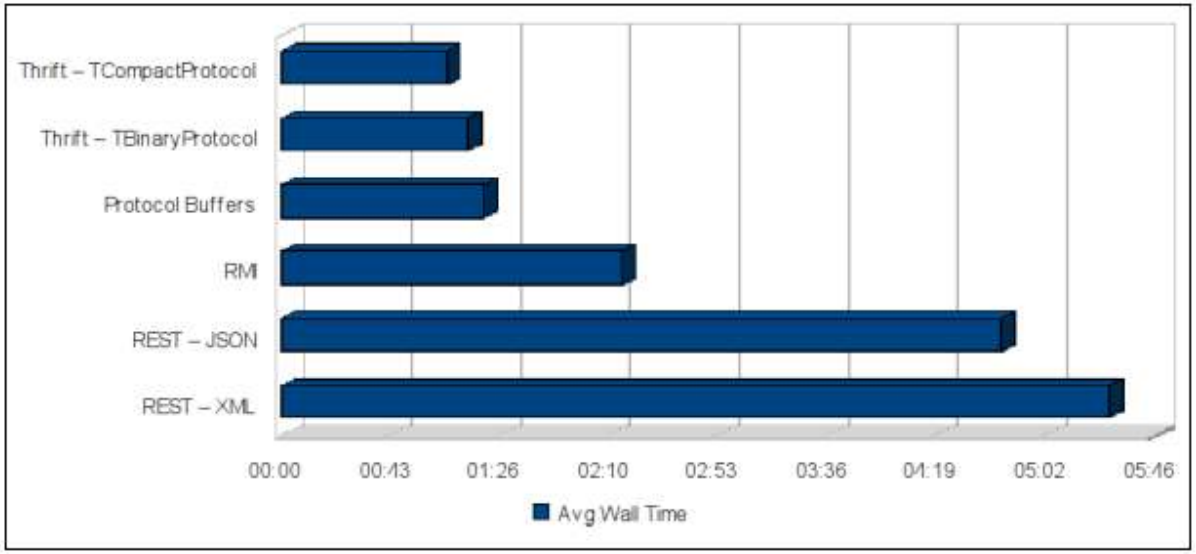
Runtime Performance

- **Test Scenario**
  - Query the list of Course numbers.
  - Fetch the course for each course number.
  - This scenario is executed 10,000 times. The tests were run on the following systems:

Operating System	Ubuntu®
CPU	Intel® Core™ 2 T5500 @ 1.66 GHz
Memory	2GiB
Cores	2

Runtime Performance





Runtime Performance

	Server CPU %	Avg. Client CPU %	Avg. Time
REST — XML	12.00%	80.75%	05:27.45
REST — JSON	20.00%	75.00%	04:44.83
RMI	16.00%	46.50%	02:14.54
Protocol Buffers	30.00%	37.75%	01:19.48
Thrift — TBinaryProtocol	33.00%	21.00%	01:13.65
Thrift — TCompactProtocol	30.00%	22.50%	01:05.12

Versioning

- The system must be able to support reading of old data, as well as requests from out-of-date clients to new servers, and vice versa.
- Versioning in Thrift and Protobuf is implemented via field identifiers.
- The combination of this field identifiers and its type specifier is used to uniquely identify the field.
- An a new compiling isn't necessary.
- Statically typed systems like CORBA or RMI would require an update of all clients in this case.

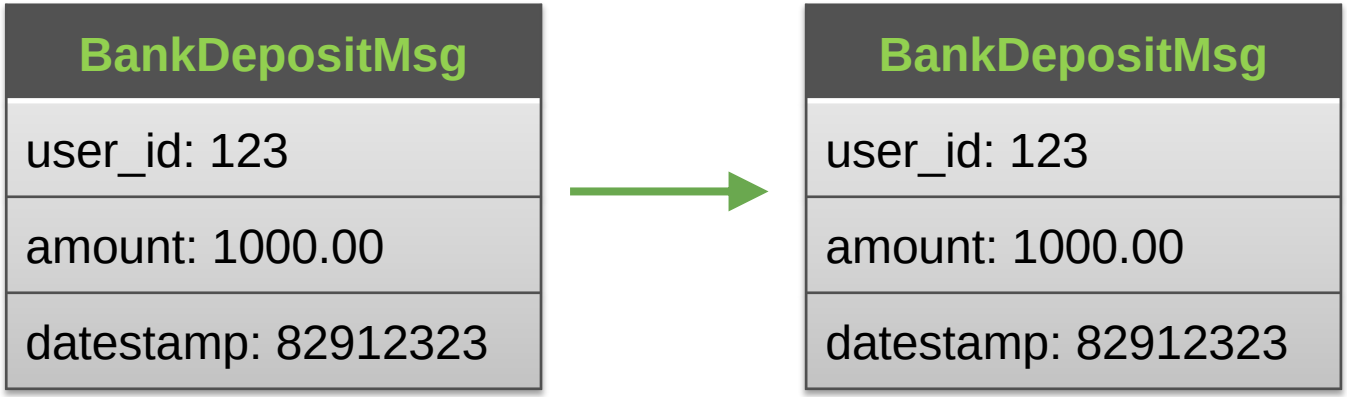
VERSION 01
VERSION 02
VERSION 03
VERSION 04
VERSION 05
VERSION 06

Forward and Backward Compatibility Case Analysis

There are four cases in which version mismatches may occur:

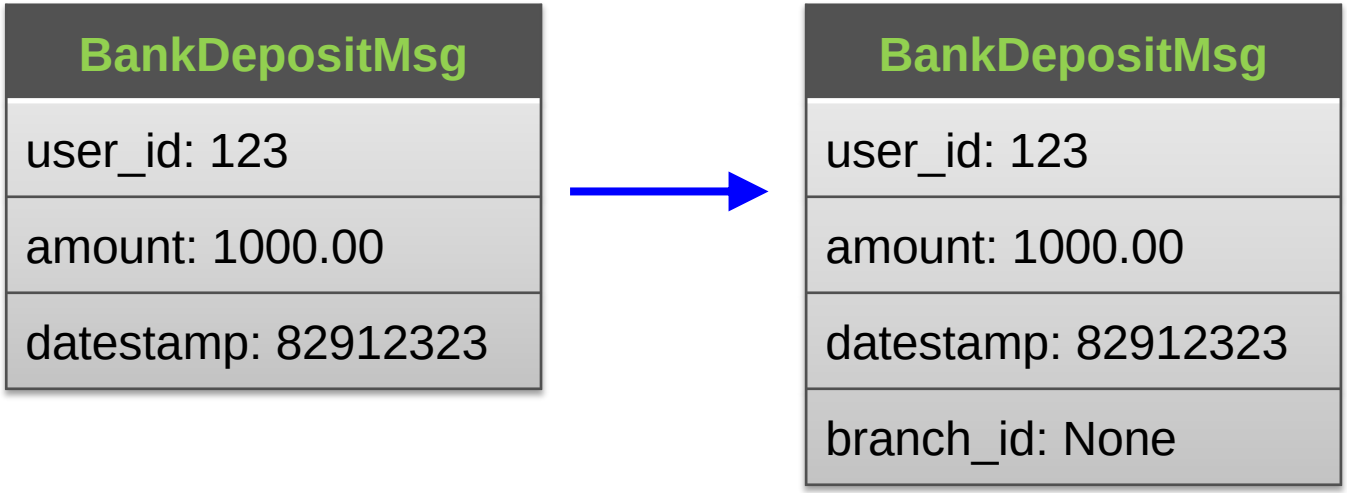
1. Added field, old client, new server.
2. Removed field, old client, new server.
3. Added field, new client, old server.
4. Removed field, new client, old server.

Forward and Backward Compatibility: Example 1



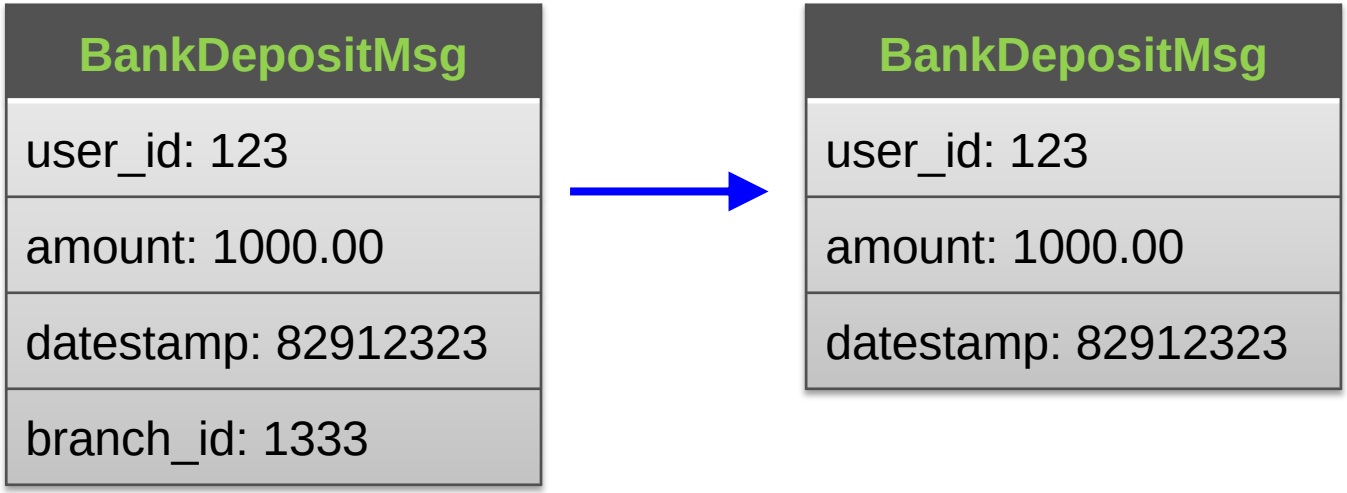
Producer (client) sends a message to a consumer (server). All good.

Forward and Backward Compatibility: Example 2



Producer (old client) sends an old message to a consumer (new server). The new server recognizes that the field is not set, and implements default behavior for out-of-date requests… Still good

Forward and Backward Compatibility: Example 3



Producer (new client) sends a new message to an consumer (old server). The old server simply ignores it and processes as normal... Still good

Serialization/deserialization performance are unlikely to be a decisive factor

	Thrift	Protocol Buffers
Features	Richer feature set, but varies from language to language	Fewer features but robust implementations
Code Quality and Design	It was open sourced by Facebook in April 2007 probably to speed up development and leverage the community's efforts.	Compare a protobuf Message definition to a thrift struct definition Compare the protobuf Java generator to the thrift Java generator
Open-ness	Apache project	Open mailing list Code base and issue tracker Google still drives development
Documentation	Severely lacking, but catching up Compare the protobuf documentation to the thrift wiki	Excellent documentation

Projects Using Thrift

- **Applications, projects, and organizations using Thrift include:**
  - Facebook
  - Cassandra project
  - Hadoop supports access to its HDFS API through Thrift bindings
  - HBase leverages Thrift for a cross-language API
  - Hypertable leverages Thrift for a cross-language API since v0.9.1.0a
  - LastFM
  - DoAT
  - ThriftDB
  - Scribe
  - Evernote uses Thrift for its public API.
  - Junkdepot

Projects Using Protobuf

- Google ☺
- ActiveMQ uses the protobuf for Message store
- Netty (protobuf-rpc)
- *I couldn't find a complete list of protobuf users anywhere ☺*

Pros & Cons

	Thrift	Protocol Buffers
Pros	More languages supported out of the box Richer data structures than Protobuf (e.g.: Map and Set) Includes RPC implementation for services	Slightly faster than Thrift when using "optimize_for = SPEED" Serialized objects slightly smaller than Thrift due to more aggressive data compression Better documentation API a bit cleaner than Thrift
Cons	Good examples are hard to find Missing/incomplete documentation	.proto can define services, but no RPC implementation is defined (although stubs are generated for you).

I'd choose **Protocol Buffers** over **Thrift**, If:

- You're only using Java, C++ or Python.
  - Experimental support for other languages is being developed by third parties but are generally not considered ready for production use
- You already have an RPC implementation
- On-the-wire data size is crucial
- The lack of any real documentation is scary to you

I'd choose **Thrift** over **Protocol Buffers,** If:

- Your language requirements are anything but Java, C++ or Python.
- You need additional data structures like Map and Set
- You want a full client/server RPC implementation built-in
- You're a good programmer that doesn't need documentation or examples ☺

Wait, what about **Avro**?

- Avro is another very recent serialization system.
- Avro relies on a schema-based system
- When Avro data is read, the schema used when writing it is always present.
- Avro data is always serialized with its schema. When Avro data is stored in a file, its schema is stored with it, so that files may be processed later by any program.
- The schemas are equivalent to protocol buffers proto files, but they do not have to be generated.
- The JSON format is used to declare the data structures.
- Official support for four languages: Java, C, C++, C#, Python, Ruby
- An RPC framework.
- Apache License 2.0

Avro IDL syntax is butt ugly and error prone

```
// Avro IDL:
    { "type": "record",
     "name": "BankDepositMsg",
     "fields" : [
       {"name": "user_id", "type": "int"},
       {"name": "amount", "type": "double", "default": "0.00"},
       {"name": "datestamp", "type": "long"}
     ]
    }

// Same Thrift IDL:
    struct BankDepositMsg {
       1: required i32 user_id;
       2: required double amount = 0.00;
       3: required i64 datestamp;
    }
```

Comparison

	Avro	Thrift and Protocol Buffer
Dynamic schema	Yes	No
Built into Hadoop	Yes	No
Schema in JSON	Yes	No
No need to compile	Yes	No
No need to declare IDs	Yes	No
Bleeding edge	Yes	No
Sexy name ☺	Yes	No

Specification

- Schema represented in one of:
  - JSON string, naming a defined type.
  - JSON object of the form:
    - {"type": "typeName" ...attributes...}
  - JSON array
- Primitive types: null, boolean, int, long, float, double, bytes, string
  - {"type": "string"}
- Complex types: records, enums, arrays, maps, unions, fixed

Comparison with other systems

- Avro provides functionality similar to systems such as Thrift, Protocol Buffers, etc.
- *Dynamic typing*: Avro does not require that code be generated. Data is always accompanied by a schema that permits full processing of that data without code generation, static datatypes, etc.
- *Untagged data*: Since the schema is present when data is read, considerably less type information need be encoded with data, resulting in smaller serialization size.
- *No manually-assigned field IDs*: When a schema changes, both the old and new schema are always present when processing data, so differences may be resolved symbolically, using field names.

Avro Hands On Review

- Q3 2012, I tested the latest Avro (1.6.3)
- It throws you a message incompatible message when you change the field name
- Serious bug, crashes w/ different versions of message (no fw/back compatibility). Emailed avro-dev@...
- Documentation is nearly non-existent and no real users. Bleeding edge, little support

Q&A